PyPSA-IEI Documentation

Integrated Energy Infrastructure Model

Open Energy Transition

None

Table of contents

1. PyPSA-IEI Documentation 6
   - 1.1 About the Model 6
   - 1.2 Quick Links 6
   - 1.3 Contributors 6
2. Installation 8
   - 2.1 Requirements 8
   - 2.2 1. Clone the Repository 8
   - 2.3 2. Download Input Data 8
   - 2.4 3. Set Up the Environment 8
   - 2.5 4. Run a Scenario 9
   - 2.6 5. Run the Evaluations 9
   - 2.7 Documentation Environment (Optional) 9
3. ⚙️ Model Functionality 10
   - 3.1 Model Functionality Overview 10
     - 3.1.1 Contents 10
   - 3.2 Costs 11
     - 3.2.1 Overview 11
     - 3.2.2 Snakemake Rule 11
     - 3.2.3 How It Works 11
     - 3.2.4 Technologies Updated in All Scenarios 12
     - 3.2.5 Additional Technologies (CE Flexibility Scenario Only) 12
     - 3.2.6 How to Modify the Cost Parameters 13
   - 3.3 Powerplant Lifetimes 14
     - 3.3.1 Overview 14
     - 3.3.2 Snakemake Rule 14
     - 3.3.3 How It Works 14
     - 3.3.4 Data Format 15
     - 3.3.5 Current Data in `powerplant_lifetime.csv` 16
     - 3.3.6 How to Modify 17
   - 3.4 Gas Network 19
     - 3.4.1 Overview 19
     - 3.4.2 Russian Import Removal 19
     - 3.4.3 TYNDP Gas Pipelines 19
     - 3.4.4 Scenario-Dependent Project Selection 20
     - 3.4.5 How to Modify the Gas Network 20

3.5 Hydrogen Network 21
3.5.1 Overview 21
3.5.2 Snakemake Rules 21
3.5.3 Data Sources 21
3.5.4 Pipe Capacity Calculation 22
3.5.5 Integration into the Network 22
3.5.6 How to Modify the Hydrogen Network 23
3.6 Industrial Energy Demand 24
3.6.1 Overview 24
3.6.2 Snakemake Rules 24
3.6.3 Demand Scenarios 24
3.6.4 Hydrogen Demand Distribution 25
3.6.5 Scaling to TransHyDe Totals 25
3.6.6 Ammonia and Methanol Imports 25
3.6.7 How to Change the TransHyDe Scenario 26
3.7 Energy Imports 27
3.7.1 Overview 27
3.7.2 Snakemake Rule 27
3.7.3 Import Technologies 27
3.7.4 Implementation 27
3.7.5 Input CSV Format 29
3.7.6 How to Modify Imports 29
3.8 Transmission Losses 30
3.8.1 Overview 30
3.8.2 Snakemake Rule 30
3.8.3 Carriers with Losses 30
3.8.4 Implementation 30
3.8.5 Configuration 31
3.8.6 How to Change Transmission Loss Values 31
3.9 Electricity Network 32
3.9.1 Overview 32
3.9.2 Snakemake Rule 32
3.9.3 Scenario Behaviour 32
3.9.4 CSV Data Format 32
3.9.5 Configuration 33
3.9.6 How to Change the TYNDP enforcement 33
3.10 Myopic Optimization (Brownfield) 35
3.10.1 Overview 35

3.10.2 Snakemake Rule 35
3.10.3 1. Carrying Forward Previous Capacity 35
3.10.4 2. Gas Pipeline Retrofitting 36
3.10.5 3. New Powerplants 36
3.10.6 4. Import Pipeline and Terminal Retrofitting 36
3.10.7 How to Configure 37
3.11 Expansion Limits 38
3.11.1 Overview 38
3.11.2 Snakemake Rule 38
3.11.3 Carriers and Components 38
3.11.4 CSV Format 39
3.11.5 Config 39
3.11.6 How to Modify the Expansion Limits 40
4. 📊 Post-Analysis 41
4.1 Post-Analysis Overview 41
4.1.1 How to Run 41
4.1.2 Output Structure 41
4.1.3 Analysis Modules 42
4.1.4 Country Scopes 42
4.2 System Costs 43
4.2.1 Overview 43
4.2.2 Output Files 43
4.2.3 Technology Mapping 44
4.2.4 Backup Capacities 44
4.3 Energy Balances 45
4.3.1 Overview 45
4.3.2 Carriers 45
4.3.3 Output Files 45
4.3.4 Configuration 46
4.4 KPIs 47
4.4.1 Overview 47
4.4.2 Installed Capacity 47
4.4.3 Infrastructure Volume (TW·km) 47
4.4.4 Summary KPIs 48
4.4.5 Case Study KPIs 48
4.4.6 Dispatch Barchart 48
4.5 Import Analysis 49
4.5.1 Overview 49

4.5.2 Import Sets 49
4.5.3 Output Files 49
4.6 Balance Maps 50
4.6.1 Overview 50
4.6.2 Carriers 50
4.6.3 Output Files 50
4.6.4 Configuration 51
5. Release Notes 52

1. PyPSA-IEI Documentation

PyPSA-IEI is an energy-system model based on the open-source PyPSA-Eur framework (release 0.10), extended to meet the needs of the study:

> "*Integrated Infrastructure Planning and 2050 Climate Neutrality: Deriving Future-Proof European Energy Infrastructures*" — Fraunhofer IEG, Fraunhofer ISI and d-fine (November 2025)
>
> Download report (PDF)

1.1 About the Model

PyPSA-IEI performs **myopic pathway optimisation** of the European energy system across multiple planning horizons between 2020 and 2050 in 5 year increments. It co-optimizes the electricity, hydrogen, gas, heat, and industry sectors within an integrated framework to identify the least-cost configuration of the overall energy system.

Key extensions on top PyPSA-Eur v0.10 include:

- TYNDP-based electricity and gas network enforcement
- Hydrogen network (European Hydrogen Backbone + ENTSO-G)
- Energy import modelling (hydrogen, syngas, synfuel)
- TransHyDe-based industrial demand
- Custom cost assumptions and powerplant lifetimes

1.2 Quick Links

- Installation — set up the environment and input data
- Model Functionality — detailed description of model extensions

1.3 Contributors

d-fine GmbH — Robert Beestermöller, Caroline Blocher, Lukas Czygan, Linus Erdmann, Felix Greven, Paula Hartnagel, Julian Hohmann, Paul Kock, Julius Meyer, Ari Pankiewicz, Anna Thünen

Fraunhofer IEG — Benjamin Pfluger, Mario Ragwitz, Caspar Schauß

Fraunhofer ISI — Khaled Al-Dabbas, Sirin Alibas, Wolfgang Eichhammer, Tobias Fleiter

For inquiries: wolfgang.eichhammer@isi.fraunhofer.de or info@d-fine.com

2. Installation

2.1 Requirements

- Conda (Miniconda or Anaconda)
- Snakemake (included in the environment)
- A supported solver: Gurobi (recommended) or HiGHS

2.2 1. Clone the Repository

```
git clone https://github.com/open-energy-transition/PyPSA-IEI.git
cd PyPSA-IEI
```

2.3 2. Download Input Data

This repository only contains input data explicitly developed for this study. To access the files provided by the open-source foundation, download the `data` folder from:

> PyPSA-Eur (February 19th 2024)

Copy these files into the `data/` folder of this repository.

Additional data will be automatically retrieved by Snakemake rules during the first run.

> **Note**
>
> The data used to generate the study results were downloaded on **June 27th 2024**.

2.4 3. Set Up the Environment

Create and activate the conda environment:

```
conda env create -f envs/environment_pypsa_iei.yaml
conda activate pypsa-iei
```

For detailed instructions on setting up PyPSA-Eur, refer to the official documentation.

2.5 4. Run a Scenario

Results for the study were generated across four main scenarios and four sensitivities. To reproduce a scenario, run Snakemake with the desired configuration file:

```
snakemake -call all --configfile config/scenarios/<scenario_file>.yaml
```

For example, to run the base scenario:

```
snakemake -call all --configfile config/scenarios/config.base.yaml
```

For further details on running PyPSA-Eur-based models, refer to the open-source documentation.

2.6 5. Run the Evaluations

To use the analysis scripts developed for this study:

1. Copy the following files into `scripts_analysis/shapes/` :
   - `country_shapes.geojson`
   - `regions_offshore_elec_s_62.geojson`
   - `regions_onshore_elec_s_62.geojson`
2. Open `scripts_analysis/analysis_main.py` and enter your specifications for the results you want to analyze.
3. Execute it:

```
python scripts_analysis/analysis_main.py
```

This will run all evaluations automatically.

2.7 Documentation Environment (Optional)

To build or preview this documentation locally, create the lightweight docs environment:

```
conda env create -f envs/environment_docs.yaml
conda activate pypsa-iei-docs
mkdocs serve
```

Then open http://127.0.0.1:8000 in your browser.

3. ⚙️ Model Functionality

3.1 Model Functionality Overview

This section describes the key extensions and modifications made to the PyPSA-Eur v0.10.0 base model for the PyPSA-IEI study. Each sub-page covers one topic with a description, the relevant Snakemake rule(s), and code/config snippets for reproducibility and future modification.

3.1.1 Contents

Topic	Description
Costs	Custom cost assumptions from config and technology-data
Lifetimes	Country- and plant-level powerplant lifetime adjustments
Gas	TYNDP gas pipelines, Russian import removal
Hydrogen	Wasserstoffkernnetz, ENTSO-G network, retrofitting
Industrial Demand	TransHyDe-based industrial energy demand
Imports	H_2, syngas, and synfuel import nodes
Electricity Network	TYNDP line enforcement and extendability
Myopic Optimization	Brownfield capacity accounting across horizons
Expansion Limits	Min/max capacity constraints per carrier and country

3.2 Costs

3.2.1 Overview

The model uses cost assumptions from technology-data v0.9.0 as a baseline (stored in `data/costs_{planning_horizon}.csv`). Selected investment costs are **overridden per planning horizon** using values defined in `config/config.agora.yaml`.

3.2.2 Snakemake Rule

Rule: `update_costs_csv`
Script: `scripts/update_costs_csv.py`

This rule reads the baseline costs CSV and updates specific technology investment costs before the network is built.

3.2.3 How It Works

scripts/update_costs_csv.py

```python
def insert_new_costs(costs_file, invest_update, filename):
    costs = pd.read_csv(costs_file).set_index('technology')
    for key, investment in invest_update.items():
        mask = (costs.parameter == "investment") & (costs.index == key)
        costs.loc[mask, 'value'] = investment
    costs.to_csv(filename)

# Called with per-horizon values from config:
investment_update = snakemake.params.invest_update[int(snakemake.wildcards.planning_horizons)]
insert_new_costs(snakemake.input.costs, investment_update, snakemake.output.costs)
```

3.2.4 Technologies Updated in All Scenarios

These investment costs are overridden for **every scenario** and every planning year:

Technology	Config key
HVDC overhead line	`HVDC overhead`
HVDC submarine cable	`HVDC submarine`
HVAC overhead line	`HVAC overhead`
H_2 electrolysis	`electrolysis`
CO_2 pipeline	`CO2 pipeline`
CO_2 submarine pipeline	`CO2 submarine pipeline`

3.2.5 Additional Technologies (CE Flexibility Scenario Only)

In the CE flexibility scenario (`config.CE_flexibility.yaml`), investment costs for the following heat-sector technologies are set to **20% above the technology-data v0.9.0 baseline** for every planning year:

Technology (config key)
`central air-sourced heat pump`
`central ground-sourced heat pump`
`decentral air-sourced heat pump`
`decentral ground-sourced heat pump`
`central solid biomass CHP`
`central gas CHP`
`micro CHP`
`biomass CHP capture`
`central water tank storage`
`decentral water tank storage`
`central resistive heater`
`decentral resistive heater`

Example for 2030 (EUR/kW unless otherwise noted):

Technology	Baseline (tech-data)	CE Flexibility (+20%)
`central air-sourced heat pump`	906.1	1087.3
`decentral air-sourced heat pump`	899.5	1079.4
`central gas CHP`	592.6	711.1
`micro CHP`	7841.7	9410.1
`central water tank storage`	0.576 EUR/kWh	0.691 EUR/kWh

3.2.6 How to Modify the Cost Parameters

Step 1: Find the correct technology name

Technology names must exactly match the `technology` column in `data/costs_<year>.csv`, which in turn comes from technology-data v0.9.0.

Step 2: Edit the scenario config file

Add or update the technology under `costs.investment` in `config/config.agora.yaml` (base config, applies to all scenarios) or in a scenario-specific file to override only for that scenario. Units depend on the technology — check the `unit` column in the costs CSV first.

config/config.agora.yaml or config/scenarios/config.*.yaml

```yaml
costs:
  investment:
    2020:
      HVDC overhead: 2000.0        # EUR/MW — IEA 2023
      HVDC submarine: 2000.0       # EUR/MW
      HVAC overhead: 500.0         # EUR/MW
      electrolysis: 1260.0         # EUR/kW — IEA Global Hydrogen Review 2023
      CO2 pipeline: 13000.0        # EUR/MW — Danish Energy Agency
      CO2 submarine pipeline: 33000.0  # EUR/MW — Danish Energy Agency
    2025:
      HVDC overhead: 2000.0
      ...
```

3.3 Powerplant Lifetimes

3.3.1 Overview

Standard powerplantmatching data does not always reflect country-specific or plant-specific decommissioning decisions. This model updates powerplant lifetimes based on a custom CSV, incorporating data from:

- Beyond Fossil Fuels
- Reuters
- Forum Energii (2024)

3.3.2 Snakemake Rule

Rule: `build_powerplants`
Script: `scripts/build_powerplants.py`
Data file: `data/powerplant_lifetime.csv`

3.3.3 How It Works

The `build_powerplants` rule retrieves the standard powerplant database from powerplantmatching and then calls `manipulate_lifetimes()` to apply corrections from the custom CSV:

scripts/build_powerplants.py

```
ppl = manipulate_lifetimes(ppl, snakemake.input.new_ppl_lifetimes)
```

The function handles four cases based on the `Name` and `status` columns:

scripts/build_powerplants.py

```
def manipulate_lifetimes(ppl_lifetime, path_lifetimes):
    lifetime_data = pd.read_csv(path_lifetimes)

    # Four adjustment categories:
    ppl_specific_lifetimes_limit  = lifetime_data.query("(Name!='all') & (status=='limit')")
    country_level_lifetimes_limit = lifetime_data.query("(Name=='all') & (status=='limit')")
    ppl_specific_lifetimes_ext    = lifetime_data.query("(Name!='all') & (status=='extend')")
    country_level_lifetimes_ext   = lifetime_data.query("(Name=='all') & (status=='extend')")

    # For 'limit': replace DateOut only if the new date is *earlier* than the existing one
    # For 'extend': replace DateOut only if the new date is *later* than the existing one
    # Applied first at country level, then at individual plant level
```

Adjustments are applied at two levels:

- **Country-level** (`Name == 'all'`): All plants of a given fuel type in a country get their `DateOut` updated (e.g. coal phase-out in Germany by 2030)
- **Plant-level** (`Name != 'all'`): Individual plants receive specific retirement years based on confirmed announcements

The logic is conservative: a `limit` entry only takes effect if the plant would have lived *longer* than specified; an `extend` entry only takes effect if the plant would have retired *earlier* than specified.

3.3.4 Data Format

`data/powerplant_lifetime.csv` columns (in order):

Column	Description
`Country`	ISO2 country code
`Fueltype`	e.g. `Lignite`, `Hard Coal`, `Nuclear`
`DateOut`	Target retirement year
`Name`	Plant name for plant-level rules; `all` for country-level rules
`status`	`limit` (force earlier retirement) or `extend` (force later retirement)
`Reference`	Data source citation

3.3.5 Current Data in `powerplant_lifetime.csv`

Coal / Lignite Phase-outs (country-level `limit`)

All entries use `Name = all` and `status = limit`, sourced from Beyond Fossil Fuels except Poland which uses Forum Energii (25.11.2024):

Country	Fueltype	DateOut
PL	Lignite + Hard Coal	2044
DE	Lignite + Hard Coal	2038
BG	Lignite + Hard Coal	2038
HR, CZ, SI	Lignite + Hard Coal	2033
ME	Lignite + Hard Coal	2035
RO	Lignite + Hard Coal	2032
MK	Lignite + Hard Coal	2030
NL, FI	Lignite + Hard Coal	2029
DK	Lignite + Hard Coal	2028
FR, HU, IT	Lignite + Hard Coal	2027
GR	Lignite + Hard Coal	2026
ES, IE	Lignite + Hard Coal	2025
SK	Lignite + Hard Coal	2024
GB	Lignite + Hard Coal	2024
PT	Lignite + Hard Coal	2021
AT, SE	Lignite + Hard Coal	2020
BE	Lignite + Hard Coal	2016

Nuclear Lifetime Extensions (plant-level `extend`)

French plants extended per Macron's 2022 announcement (Reuters), Belgian reactors extended per the 2023 Engie agreement (Reuters):

Country	Plant	DateOut
FR	Civaux	2057
FR	Nogent, Belleville	2047
FR	Golfech, Penly	2050
FR	St Alban, Flamanville	2045
FR	Cruas	2043
FR	Paluel	2044
FR	Cattenom	2038
FR	Blayais, Gravelines, Dampierre	2040–2041
FR	Chooz	2027
FR	Bugey	2032
FR	Chinon	2023
BE	Doel, Tihange	2035

3.3.6 How to Modify

Limit a country-wide fuel type (phase-out)

Set `Name` to `all` and `status` to `limit`. All plants of that fuel type in that country will retire no later than `DateOut`:

```
Country,Fueltype,DateOut,Name,status,Reference
DE,Lignite,2030,all,limit,custom
PL,Hard Coal,2040,all,limit,custom
```

Extend a country-wide fuel type (lifetime extension)

```
Country,Fueltype,DateOut,Name,status,Reference
CZ,Nuclear,2050,all,extend,custom
```

Override a specific plant

Use the exact plant name as it appears in the powerplantmatching database:

```
Country,Fueltype,DateOut,Name,status,Reference
PL,Lignite,2036,Bełchatów,limit,custom
CZ,Nuclear,2050,Dukovany,extend,custom
```

Disable lifetime corrections entirely

Leave the `new_ppl_lifetimes` input empty in the Snakemake config (the function returns the original dataframe unchanged if no path is provided).

3.4 Gas Network

3.4.1 Overview

Two key modifications are made to the gas network relative to base PyPSA-Eur:

1. **Russian gas imports are removed** in all scenarios by default
2. **TYNDP gas pipeline projects** are processed and added to the model

3.4.2 Russian Import Removal

Rule: `build_gas_input_locations`
Script: `scripts/build_gas_input_locations.py`

The base config sets `import_from_russia: false` under `sector:`, so Russian pipeline entry points are excluded in all scenarios. If re-enabled but `nordstream` is disabled, only the Nord Stream pipeline is excluded:

scripts/build_gas_input_locations.py

```
if not snakemake.params.import_from_russia:
    entry = entry.loc[~(entry.from_country == "RU")]
elif not snakemake.params.nordstream:
    entry = entry.loc[entry.id != "INET_BP_63"]
```

3.4.3 TYNDP Gas Pipelines

Rule: `build_tyndp_gas_pipes`
Script: `scripts/build_tyndp_gas_pipes.py`

TYNDP gas pipeline project data is processed and mapped to model cluster regions using geographic coordinates:

scripts/build_tyndp_gas_pipes.py

```
# Load TYNDP project list
tyndp_raw_data = pd.read_excel(snakemake.input.tyndp_gas_projects)

# Remove projects with missing critical data
tyndp_raw_data.dropna(subset=[
    'Code', 'Project Name', 'Maturity Status', 'Diameter (mm)',
    'Length (km)', 'PCI 5th List', 'Project Commissioning Year Last',
    'Start', 'End'
], inplace=True)

# Convert DMS coordinates to cluster names
def coordinates_to_cluster(coordinates):
    ...
    point = Point(dd_lon, dd_lat)
    return regions.loc[regions['geometry'].contains(point), 'name'].values[0]
```

Pipe capacity is derived from diameter using the `diameter_to_capacity` function (from `build_gas_network.py`). Projects entirely within one cluster are dropped. The output is saved to CSV for use in `prepare_sector_network`.

3.4.4 Scenario-Dependent Project Selection

TYNDP projects are filtered by maturity status via the scenario config:

Scenario	`allowed_statuses`
CE	`FID` only
SE	all statuses (no filter)

config/scenarios/config.CE.yaml

```yaml
policy_plans:
  include_tyndp_gas:
    enable: true
    allowed_statuses:
      - FID  # Final Investment Decision
```

3.4.5 How to Modify the Gas Network

Change which TYNDP projects are included

Set `allowed_statuses` in the scenario config file. To allow all maturity levels, omit the key entirely:

config/scenarios/config.CE.yaml

```yaml
policy_plans:
  include_tyndp_gas:
    enable: true
    allowed_statuses:
      - FID
      - Advanced
      - Less Advanced
```

Control Russian import handling

Override the parameter in the scenario config:

```yaml
sector:
  import_from_russia: false  # set to true to allow Russian pipeline gas
  nordstream: false          # set to true to include Nord Stream specifically
```

To add custom gas pipeline projects, extend the TYNDP Excel input file (`data/gas_network/TYNDP_Gas_Interconnectors.xlsx`) with the required columns (`Code`, `Diameter (mm)`, `Length (km)`, `Start`, `End`, `Maturity Status`, etc.).

3.5 Hydrogen Network

3.5.1 Overview

The hydrogen network is modelled in the **SN** and **SE** scenarios. It combines the German *Wasserstoffkernnetz* (FNB-Gas) with the European hydrogen infrastructure map, enabling both retrofitting of existing gas pipelines and construction of new H_2 pipelines.

The scripts are adapted from PyPSA-Ariadne.

3.5.2 Snakemake Rules

Rule	Script	Purpose
`build_wasserstoff_kernnetz`	`scripts/build_wasserstoff_kernnetz.py`	Process and merge FNB-Gas + ENTSO-G data
`cluster_wasserstoff_kernnetz`	`scripts/cluster_wasserstoff_kernnetz.py`	Map H_2 network to model clusters
`modify_prenetwork`	`scripts/modify_prenetwork.py`	Integrate H_2 pipelines into the network

3.5.3 Data Sources

Source	File	Geographic scope
FNB-Gas	`Anlage2_Leitungsmeldungen_...xlsx` — pipeline submissions from potential operators	Germany
FNB-Gas	`Anlage3_FNB_Massnahmenliste_...xlsx` — FNB measure list (retrofit + new build)	Germany
ENTSO-G	`data/wasserstoff_kernnetz/h2_inframap_entsog.geojson`	Europe

The two FNB-Gas files are downloaded automatically by Snakemake at runtime from fnb-gas.de. Both datasets are merged in `cluster_wasserstoff_kernnetz`. Duplicate entries between the two sources are removed to prevent double-counting of pipeline capacities.

3.5.4 Pipe Capacity Calculation

H_2 pipe capacity is estimated from pipe diameter using piecewise linear interpolation across three reference points:

scripts/build_wasserstoff_kernnetz.py

```
def diameter_to_capacity_h2(pipe_diameter_mm):
    """
    20 inch (500 mm)  50 bar ->  1.2 GW H2 (LHV)
    36 inch (900 mm)  50 bar ->  4.7 GW H2 (LHV)
    48 inch (1200 mm) 80 bar -> 16.9 GW H2 (LHV)
    """
    m0 = (1200 - 0) / (500 - 0)
    m1 = (4700 - 1200) / (900 - 500)
    m2 = (16900 - 4700) / (1200 - 900)

    if pipe_diameter_mm < 500:
        return m0 * pipe_diameter_mm
    elif pipe_diameter_mm < 900:
        return m1 * pipe_diameter_mm + (1200 - m1 * 500)
    else:
        return m2 * pipe_diameter_mm + (4700 - m2 * 900)
```

For retrofitted pipes, the displaced gas capacity is also tracked using `diameter_to_capacity` (from `build_gas_network.py`) to reduce the gas network capacity accordingly.

3.5.5 Integration into the Network

H_2 pipeline data is integrated into the network in the `modify_prenetwork` rule via `add_wasserstoff_kernnetz()`. Only pipelines with `build_year` within the current investment period are added. The function also:

- **Reduces gas pipeline capacity** (`p_nom`) on routes where kernnetz pipelines have been retrofitted from gas
- **Reduces H_2 retrofitting potential** (`p_nom_max` on H_2 pipeline retrofitted links) for all kernnetz pipelines built to date

scripts/modify_prenetwork.py

```
# Reduce gas pipeline p_nom for retrofitted kernnetz pipes
gas_pipes = n.links.query("carrier == 'gas pipeline'")
res_gas_pipes = reduce_capacity(gas_pipes, add_reversed_pipes(wkn_to_date), carrier="gas")
n.links.loc[n.links.carrier == "gas pipeline", "p_nom"] = res_gas_pipes["p_nom"]

# Reduce H2 retrofitting potential for all kernnetz pipes built to date
res_h2_pipes_retrofitted = reduce_capacity(
    h2_pipes_retrofitted,
    add_reversed_pipes(wkn),
    carrier="H2",
    target_attr="p_nom_max",
    conversion_rate=snakemake.config["sector"]["H2_retrofit_capacity_per_CH4"],
)
```

3.5.6 How to Modify the Hydrogen Network

The Wasserstoffkernnetz

The Wasserstoffkernnetz integration is controlled under `policy_plans` in the scenario config:

config/scenarios/config.SE.yaml

```yaml
policy_plans:
  wasserstoff_kernnetz:
    enable: true
    optimize_after: 2040  # pipelines are non-extendable until this year
```

Gas-to-H_2

Pipeline retrofitting is controlled separately under `sector:`:

config/config.agora.yaml

```yaml
sector:
  H2_retrofit: true
  H2_retrofit_capacity_per_CH4: 0.6  # capacity ratio H2/CH4
```

3.6 Industrial Energy Demand

3.6.1 Overview

Industry demand is updated from the standard PyPSA-Eur values using data from the TransHyDe project. Two modifications are made relative to base PyPSA-Eur:

1. **Carrier ratios** for high-temperature processes are remapped to hydrogen to better reflect intra-country spatial distribution
2. **Absolute country totals** are scaled to match TransHyDe demand values

3.6.2 Snakemake Rules

Rule	Script	Purpose
`build_industry_sector_ratios`	`scripts/build_industry_sector_ratios.py`	Set carrier ratios per industry sector
`build_industrial_energy_demand_per_node`	`scripts/build_industrial_energy_demand_per_node.py`	Scale absolute demands to TransHyDe totals

3.6.3 Demand Scenarios

Three demand scenarios are defined in `build_industry_sector_ratios.py`:

`demand_scenario`	Description
`original`	Reverts to base PyPSA-Eur v0.10.0 carrier ratios
`high-temperature`	Hydrogen replaces fossil fuels in furnaces and smelters — reflects TransHyDe Scenario 1.5 (Mid Demand)
`high-temperature+steam`	As above, plus hydrogen replaces steam processing — reflects TransHyDe Scenario 2 (High Demand)

The scenario is selected via `transhyde_scenario` in the config:

config/config.agora.yaml

```yaml
industry:
  transhyde_data: true
  transhyde_scenario: 1.5  # use "high-temperature" ratios
```

config/scenarios/config.CE_TranshydeS2.yaml

```yaml
industry:
  transhyde_scenario: 2   # use "high-temperature+steam" ratios
```

3.6.4 Hydrogen Demand Distribution

When `transhyde_data: true`, carrier ratios for high-temperature industrial processes (furnaces, smelters) are remapped to hydrogen. This only affects **spatial distribution** within a country — the absolute country totals are overwritten in the next step. From the script docstring:

> When `transhyde_data` is True, processes (mostly heat) are mapped from originally electricity/biomass/gas to hydrogen. The mapping is not precise; the aim is to get a more realistic distribution of the demand within a country.

3.6.5 Scaling to TransHyDe Totals

`build_industrial_energy_demand_per_node` reads TransHyDe CSV data and scales PyPSA-Eur's country-level totals to match. The scaling preserves the intra-country distribution from `build_industry_sector_ratios`:

scripts/build_industrial_energy_demand_per_node.py

```python
# Calculate per-carrier scaling factor: TransHyDe total / PyPSA total
factor = (demand_th_p / nodal_df_grp_m).fillna(1.0)

# Apply factor to all nodes, preserving intra-country distribution
nodal_df *= factor

# Where PyPSA has zero demand but TransHyDe has non-zero,
# distribute TransHyDe total using node weights
```

The TransHyDe data files are in `data/transhyde/`.

3.6.6 Ammonia and Methanol Imports

The model can reduce industrial energy demand when ammonia or methanol are imported rather than produced domestically. Both are disabled by default:

config/config.agora.yaml

```yaml
sector:
  ammonia: false
  ammonia_import: false
  ammonia_import_share: 50   # % of demand met by imports, if enabled
  methanol_import: false
  methanol_import_share: 50  # % of demand met by imports, if enabled
```

3.6.7 How to Change the TransHyDe Scenario

Set `transhyde_scenario` in the scenario config:

```
industry:
  transhyde_scenario: 1.5   # default (Mid Demand)
  # transhyde_scenario: 2   # High Demand
```

3.7 Energy Imports

3.7.1 Overview

Energy imports are modelled as **extendable generators** (with supporting buses and links where needed), covering five import technologies: hydrogen (pipeline and shipping), syngas (pipeline and shipping), and synfuel. Import capacities, costs, and node assignments are loaded from a planning-year-specific CSV file derived from the TransHyDe project.

3.7.2 Snakemake Rule

Rule: `prepare_sector_network`
Script: `scripts/prepare_sector_network.py` — function `add_import()`
Data: `data/import_nodes_tech_manipulated_s_62_<year>.csv`

3.7.3 Import Technologies

Technology	Components added	Carrier
`pipeline-H2`	Generator → `{node} H2` bus	`import pipeline-H2`
`shipping-H2`	Generator → `{node} H2` bus	`import shipping-H2`
`pipeline-syngas`	Bus (`syngas`) + Generator + Link (`syngas→gas`)	`import pipeline-syngas`
`shipping-syngas`	Generator → existing `syngas` bus + Link (`syngas→gas`)	`import shipping-syngas`
`synfuel`	Bus (`synfuel`) + Generator + Link (`synfuel→EU oil`)	`synfuel`

For H_2 (pipeline and shipping), only a generator is needed since H_2 buses already exist. For syngas and synfuel, an intermediate bus is created and a link converts to the model's gas/oil network with a **negative CO_2 efficiency** on the `co2 atmosphere` bus — this cancels out the emissions that would otherwise be attributed when the synthetic fuel is eventually consumed.

3.7.4 Implementation

H_2 (pipeline and shipping) — generator only, injecting directly into the node's existing H_2 bus. Shipping routes include `shipping_idx` in the component name to distinguish multiple routes to the same node:

scripts/prepare_sector_network.py

```
# pipeline-H2: generator → {node} H2
n.madd(
```

```
    "Generator",
    [f"{node} {tech}" for node, tech in zip(df_pipeline_h2["bus"], df_pipeline_h2["tech"])],
    bus=[f"{node} H2" for node in df_pipeline_h2["bus"]],
    carrier=[f"import {tech}" for tech in df_pipeline_h2["tech"]],
    p_nom_extendable=True,
    p_nom_max=df_pipeline_h2["p_nom"],
    marginal_cost=df_pipeline_h2["marginal_cost"],
    capital_cost=df_pipeline_h2["capital_cost"],
    p_min_pu=0,
    lifetime=25,
)

# shipping-H2: same structure, shipping_idx appended to name
n.madd(
    "Generator",
    [f"{node} {tech} ({idx})" for node, tech, idx in zip(
        df_shipping_h2["bus"], df_shipping_h2["tech"], df_shipping_h2["shipping_idx"])],
    bus=[f"{node} H2" for node in df_shipping_h2["bus"]],
    ...
)
```

Syngas (pipeline and shipping) — an intermediate `syngas` bus is created per node, then a link converts syngas → gas with `efficiency2=-CO2_intensity` on the `co2 atmosphere` bus:

scripts/prepare_sector_network.py

```
# syngas bus
n.madd("Bus", [f"{node} syngas" for node in syngas_nodes], carrier="syngas", unit="MWh_LHV")

# generator → syngas bus
n.madd("Generator", ..., bus=[f"{node} syngas" for node in ...])

# link: syngas → gas, with negative CO2 efficiency to cancel combustion emissions
n.madd(
    "Link",
    [f"{node} syngas" for node in syngas_nodes],
    bus0=[f"{node} syngas" for node in syngas_nodes],
    bus1=[f"{node} gas" for node in syngas_nodes],
    bus2="co2 atmosphere",
    carrier="syngas to gas",
    efficiency2=-costs.at["gas", "CO2 intensity"],
    p_nom_extendable=True,
)
```

Synfuel — same pattern: `synfuel` bus → generator → link to `EU oil` with `efficiency2=-costs.at["oil", "CO2 intensity"]`.

3.7.5 Input CSV Format

`data/import_nodes_tech_manipulated_s_62_<year>.csv`:

Column	Description
`bus`	Model cluster node (e.g. `DE0 0`) or `EU` for European aggregate
`tech`	Import technology string
`p_nom`	Maximum import capacity [MW]
`marginal_cost`	Variable cost [EUR/MWh]
`capital_cost`	Annualised investment cost [EUR/MW/year]
`shipping_idx`	Unique index for shipping routes

3.7.6 How to Modify Imports

Import Capacities and Costs

To change import capacities or costs, edit the relevant year file:

```
data/import_nodes_tech_manipulated_s_62_2030.csv
data/import_nodes_tech_manipulated_s_62_2035.csv
...
```

Import nodes

To add a new import node, add a row with the correct bus name, technology, and capacity. Bus names must match nodes in the clustered network (`s_62` = 62-node clustering). To disable all imports, remove or comment out the `add_import(n)` call in `prepare_sector_network.py`.

3.8 Transmission Losses

3.8.1 Overview

Transmission losses are modelled by splitting each bidirectional link into two unidirectional links with `efficiency < 1.0`. The function `lossy_bidirectional_links()` is applied to every carrier listed under `sector.transmission_efficiency` in the config. DC links carry a static converter loss and a length-dependent resistive loss. H_2 and gas pipelines carry compression losses, drawn as electricity from the sending node. TYNDP gas pipelines receive the same loss parameters as standard gas pipelines.

3.8.2 Snakemake Rule

Rule: `prepare_sector_network`
Script: `scripts/prepare_sector_network.py` — function `lossy_bidirectional_links()`

3.8.3 Carriers with Losses

Carrier	`efficiency_static`	`efficiency_per_1000km`	`compression_per_1000km`
DC	0.98	0.977	—
H2 pipeline	—	1.0	0.019
gas pipeline	—	1.0	0.010
electricity distribution grid	1.0	—	—

3.8.4 Implementation

Each bidirectional link is split into a forward link (efficiency applied by length and static factor) and a reversed copy (`capital_cost = 0`, `length = 0`). For H_2 and gas pipelines, a secondary electricity bus (`bus2`) is attached at the sending node and `efficiency2` is set to a negative value proportional to pipeline length, representing compression energy consumption.

The function is called in a loop over all configured carriers:

scripts/prepare_sector_network.py

```
for k, v in options["transmission_efficiency"].items():
    lossy_bidirectional_links(n, k, v)
```

TYNDP gas pipelines receive an additional dedicated call using the same `gas pipeline` efficiencies:

scripts/prepare_sector_network.py

```
losses = snakemake.config["sector"]["transmission_efficiency"]["gas pipeline"]
lossy_bidirectional_links(n, "gas pipeline tyndp", losses)
```

3.8.5 Configuration

Efficiency parameters are set under `sector.transmission_efficiency`:

config/config.agora.yaml

```yaml
sector:
  transmission_efficiency:
    DC:
      efficiency_static: 0.98
      efficiency_per_1000km: 0.977
    H2 pipeline:
      efficiency_per_1000km: 1      # 0.979 commented out
      compression_per_1000km: 0.019
    gas pipeline:
      efficiency_per_1000km: 1      # 0.977 commented out
      compression_per_1000km: 0.01
    electricity distribution grid:
      efficiency_static: 1          # 0.97 commented out
```

Note

`solving.options.transmission_losses` does **not** control whether losses are applied. It only triggers removal of AC lines with `num_parallel == 0` that would conflict with the loss-aware solver:

```yaml
solving:
  options:
    transmission_losses: 2
```

3.8.6 How to Change Transmission Loss Values

Edit the efficiency values under `sector.transmission_efficiency` in `config/config.agora.yaml` or a scenario override file. Setting `efficiency_per_1000km: 1` and `compression_per_1000km: 0` for a carrier effectively disables losses for that carrier.

3.9 Electricity Network

3.9.1 Overview

TYNDP electricity transmission projects are enforced on the network in `prepare_sector_network`. Per-project capacity limits are read from two CSV files and applied for each planning horizon. Cross-border lines and links can optionally be made extendable from a given year onwards.

3.9.2 Snakemake Rule

Rule: `prepare_sector_network`
Script: `scripts/prepare_sector_network.py` — function `set_capacity_tyndp_elec()`

Input data:

- `data/links_2020-2050.csv` — DC link capacity limits per year
- `data/lines_2020-2050.csv` — AC line capacity additions per year and TYNDP status

3.9.3 Scenario Behaviour

Scenario	TYNDP statuses included	Cross-border extendable from
CE, CN	`under construction`	All horizons (`optimize_after: true`)
SE, SN	`in planning`, `in permitting`, `under construction`	2040 onwards (`optimize_after: 2040`)

3.9.4 CSV Data Format

`data/links_2020-2050.csv`

Columns: `name`, `bus0`, `bus1`, `p_nom`, `2020`, `2025`, ..., `2050`

Each year column contains a fraction of `p_nom` or `opt`:

Value	Effect
`0.0`	Link not active in this year
`1` (float > 0)	`p_nom_min = p_nom_max = p_nom × value`, link made extendable
`opt`	`p_nom_min = p_nom`, link remains extendable without upper bound

`data/lines_2020-2050.csv`

Columns: `name`, `bus0`, `bus1`, then one column per `{year}{status}` combination (e.g., `2025under construction`, `2030in planning`).

Each cell holds the capacity addition [MW] for that year and status, or `opt` for a soft minimum (extendable without upper bound). Values with an `opt` suffix set the minimum and leave the line extendable.

3.9.5 Configuration

TYNDP electricity enforcement is disabled by default and enabled per scenario:

config/config.agora.yaml

```yaml
policy_plans:
  include_tyndp_elec:
    enable: false           # set to true in scenario configs
    allowed_statuses:
      - under consideration
      - in planning
      - in permitting
      - under construction
    optimize_after: true    # true = all horizons; integer year = from that year
```

config/scenarios/config.CE.yaml

```yaml
policy_plans:
  include_tyndp_elec:
    enable: true
    allowed_statuses:
      - under construction
    # optimize_after inherits true from base config
```

config/scenarios/config.SE.yaml

```yaml
policy_plans:
  include_tyndp_elec:
    enable: true
    allowed_statuses:
      - in planning
      - in permitting
      - under construction
    optimize_after: 2040
```

3.9.6 How to Change the TYNDP enforcement

Configure which TYNDP projects are enforced

Edit `policy_plans.include_tyndp_elec.allowed_statuses` in the scenario config. Only projects whose TYNDP status tag matches an entry in this list are enforced.

Configure when cross-border links become extendable

Set `policy_plans.include_tyndp_elec.optimize_after` to `true` (all horizons) or to an integer year (e.g., `2040`) in the scenario config.

Modify apacity values per horizon

Edit the year columns in `data/links_2020-2050.csv` (DC links) or the `{year}{status}` columns in `data/lines_2020-2050.csv` (AC lines).

3.10 Myopic Optimization (Brownfield)

3.10.1 Overview

Myopic pathway optimization builds each planning horizon on top of the previous one. The `add_brownfield` rule manages this transition by:

1. Carrying forward previously built capacity (fixed, non-extendable)
2. Accounting for gas pipeline retrofitting to H_2
3. Adding newly commissioned powerplants
4. Updating import pipeline and terminal retrofit potentials

3.10.2 Snakemake Rule

Rule: `add_brownfield`
Script: `scripts/add_brownfield.py` — function `add_brownfield()`

3.10.3 1. Carrying Forward Previous Capacity

All assets (Generator, Link, Store) from the previous horizon that are still within their lifetime are imported into the current network with their **optimized capacity fixed** (non-extendable):

scripts/add_brownfield.py

```
for c in n_p.iterate_components(["Link", "Generator", "Store"]):
    # Remove assets that have expired
    n_p.mremove(c.name, c.df.index[c.df.build_year + c.df.lifetime < year])

    # Remove assets below minimum capacity threshold
    n_p.mremove(c.name, c.df.index[
        c.df[f"{attr}_nom_extendable"] & (c.df[f"{attr}_nom_opt"] < threshold)
    ])

    # Fix capacity at optimized value from previous horizon
    c.df[f"{attr}_nom"] = c.df[f"{attr}_nom_opt"]
    c.df[f"{attr}_nom_extendable"] = False

    n.import_components_from_dataframe(c.df, c.name)
```

3.10.4 2. Gas Pipeline Retrofitting

When `H2_retrofit: true`, existing gas pipeline capacity is reduced by the amount already retrofitted to H_2 in previous horizons:

scripts/add_brownfield.py

```
CH4_per_H2 = 1 / snakemake.params.H2_retrofit_capacity_per_CH4

# Already retrofitted H2 capacity from previous years
already_retrofitted = (
    n.links.loc[h2_retrofitted_fixed_i, "p_nom"]
    .rename(lambda x: x.split("-2")[0].replace(fr, to) + f"-{year}")
    .groupby(level=0).sum()
)

# Remaining gas capacity = original - retrofitted (adjusted for energy ratio)
remaining_capacity = (
    pipe_capacity - CH4_per_H2
    * already_retrofitted.reindex(pipe_capacity.index).fillna(0)
)
n.links.loc[gas_pipes_i, "p_nom"] = remaining_capacity
n.links.loc[gas_pipes_i, "p_nom_max"] = remaining_capacity
```

3.10.5 3. New Powerplants

A base powerplant network is loaded and newly commissioned plants are added for the current horizon:

scripts/add_brownfield.py

```
n_ppl = pypsa.Network(snakemake.input.base_powerplants)
all_years = snakemake.config["scenario"]["planning_horizons"]
previous_year = str(all_years[all_years.index(year) - 1])

for ppl_component in n_ppl.iterate_components(["Link", "Generator"]):
    # Select plants commissioned between previous and current year
    idx_ppl = ppl_component.df.index[
        ends_with_year(ppl_component.df.index)
        & (ppl_component.df.index.str[-4:] > previous_year)
        & (ppl_component.df.index.str[-4:] <= str(year))
    ]
    n.generators.loc[idx_ppl, 'p_nom'] = ppl_component.df.loc[idx_ppl, 'p_nom']
```

3.10.6 4. Import Pipeline and Terminal Retrofitting

The same retrofit logic is applied to non-European gas import pipelines and LNG terminals:

scripts/add_brownfield.py

```
# Remaining gas pipeline capacity after retrofit
remaining_capacity_pipes = (
    full_capacity_pipes
    - CH4_per_H2 * retrofitted_capacity_pipes
      .rename(lambda x: x[:5] + ' import pipeline gas')
      .groupby(level=0).sum()
)
n.generators.loc[filtered_natural_gas_pipes_i, 'p_nom_max'] = remaining_capacity_pipes

# Remaining H2 retrofit potential
```

```
remaining_capacity_pipes_retro = (
    max_capacity_retrofit_pipes
    - retrofitted_capacity_pipes
      .rename(lambda x: x[:-4] + str(year))
      .groupby(level=0).sum()
)
n.generators.loc[filtered_h2_pipes_ext_i, 'p_nom_max'] = remaining_capacity_pipes_retro
```

The same pattern applies to LNG → H_2 terminal retrofitting.

3.10.7 How to Configure

config/config.agora.yaml

```yaml
sector:
  H2_retrofit: true                      # enable gas-to-H2 retrofitting
  H2_retrofit_capacity_per_CH4: 0.6      # H2 capacity per CH4 capacity unit

existing_capacities:
  threshold_capacity: 10                 # MW — assets below this are dropped
```

Planning horizons are defined in the scenario config:

config/scenarios/config.CE.yaml

```yaml
scenario:
  planning_horizons: [2020, 2025, 2030, 2035, 2040, 2045, 2050]
```

3.11 Expansion Limits

3.11.1 Overview

Minimum and maximum capacity constraints are enforced for selected renewable and other carriers across European countries for each planning year. The constraints target **total cumulative installed capacity** per country and carrier. Existing (non-extendable) capacity is subtracted before the bound is applied to the extendable `p_nom` decision variable.

3.11.2 Snakemake Rule

Script: `scripts/solve_network.py` — function `add_country_carrier_limit_constraints`, called from `extra_functionality`
Rule: `solve_sector_network_myopic`
Data files: selected via `policy_plans.agg_p_nom_limits` in the scenario config

Scenario	Data file	Min/max band
CE, SE, and variants	`data/agg_p_nom_minmax_european.csv`	±20 %
CN, SN	`data/agg_p_nom_minmax_national.csv`	±15 %

3.11.3 Carriers and Components

Carrier	Component	Geographic scope	Years with data
`solar`	Generator	All modelled countries	2030, 2050
`onwind`	Generator	All modelled countries	2030, 2050
`offwind`	Generator	All modelled countries	2030, 2050
`H2 Electrolysis`	Link	European-wide (`Eur`) + Romania	2030
`gas for electricity`	Link	Poland (2020–2030), Romania (2020–2050)	varies
`co2 sequestered`	Store	Romania	2030–2050

`gas for electricity` is a **grouped carrier**: OCGT, CCGT, urban central gas CHP (and CHP CC), micro gas CHP variants, and `allam` are all counted together against this limit. For Links, the constraint is applied to output capacity (`p_nom × efficiency`), not input capacity.

3.11.4 CSV Format

Both files share the same long format: one row per (country, carrier, year) combination. Empty `min` or `max` cells mean no constraint for that direction.

Column	Description
`country`	ISO2 country code, or `Eur` for European-wide aggregate
`carrier`	Carrier name as it appears in the constraint (see grouping above)
`year`	Planning year (integer)
`min`	Minimum total installed capacity [MW or t for CO_2] — blank = unconstrained
`max`	Maximum total installed capacity [MW or t for CO_2] — blank = unconstrained
`unit`	Unit string (e.g. `MW`, `MW_H2`, `t`)
`Source`	Source reference
`component`	PyPSA component type in **lowercase** (`generator`, `link`, `store`)

Example rows:

```
country,carrier,year,min,max,unit,Source,component
DE,solar,2030,172000.0,258000.0,MW,Ember,generator
DE,solar,2050,9600.0,175423.9,MW,Atlite,generator
PL,gas for electricity,2025,,5500.0,MW,Forum Energii 2025/03/07,link
Eur,H2 Electrolysis,2030,46978.8,111532.4,MW_H2,IEA hydrogen map,link
RO,co2 sequestered,2030,,10000000.0,t,NZIA communicated by EPG,store
```

3.11.5 Config

```yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_european.csv  # default base config
```

Scenario configs override this key:

```yaml
# config.CE.yaml / config.SE.yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_european.csv

# config.CN.yaml / config.SN.yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_national.csv
```

> **Note**
>
> The two files differ only in the ±% band applied to the 2030 Ember/NECP targets: ±20 % in the European file and ±15 % in the national file. The 2050 upper bounds (Atlite technical potentials) and all non-renewable entries are identical.

3.11.6 How to Modify the Expansion Limits

Change a capacity limit for a country

Edit the relevant CSV directly. Each row is a single (country, carrier, year) entry. For example, to raise the solar minimum in Spain for 2030:

```
ES,solar,2030,130000.0,195000.0,MW,Custom,generator
```

Add a constraint to new country or carrier

Add a new row with `country`, `carrier`, `year`, `min` / `max`, and `component`. Make sure `carrier` matches the grouped name (e.g. use `offwind`, not `offwind-ac`) and `component` is lowercase.

Add a European-wide constraint

Use `Eur` as the `country` value. The code automatically duplicates every component into an `Eur` group and adds both country-level and European-level `p_nom` to the left-hand side of the constraint.

4. 📊 Post-Analysis

4.1 Post-Analysis Overview

The post-analysis scripts are located in `scripts_analysis/` and are run **after** the PyPSA optimization is complete. The master script `analysis_main.py` orchestrates all analyses and produces plots and Excel files organized into timestamped output folders.

4.1.1 How to Run

```
cd scripts_analysis
python analysis_main.py
```

The script must be run from inside `scripts_analysis/` — `main_dir` is resolved automatically as the parent of the current working directory.

4.1.2 Output Structure

```
analysis_results_YYYYMMDD/
  ExogenousDemand/      ← transport demand plots
  Balances/             ← energy balance charts per carrier
  network_plots/        ← transmission network maps
  KPIs/                 ← installed capacity, GWkm, dispatch, summary KPIs
  self-sufficiency/     ← regional energy self-sufficiency
  costs/                ← total system cost breakdown
  utilization/          ← transmission line utilization
  Import/               ← energy import volumes and costs
  balance_maps/         ← spatial energy balance maps
```

4.1.3 Analysis Modules

Output folder	Script	Function
`ExogenousDemand/`	`exogenous_demand_analyses.py`	`get_transport_demand_plot()`
`Balances/`	`configurable_energy_balances.py`	`get_standard_balances()`
`network_plots/`	`plot_elec_network.py`	`plot_map_elec_years()`
`network_plots/`	`plot_h2_network.py`	`plot_h2_map_years()`
`network_plots/`	`plot_co2_network.py`	`plot_co2_map_years()`
`network_plots/`	`plot_ch4_network.py`	`plot_ch4_map_years()`
`network_plots/`	`compare_grids.py`	`plot_grid_comparisons()`
`KPIs/`	`installed_capacity.py`	`call_installed_capacity_plot()`
`KPIs/`	`plot_GWkm.py`	`plot_TWkm_all_carriers()`
`KPIs/`	`summary_KPIs.py`	`start_KPI_analysis()`
`KPIs/`	`plot_cases_KPIs.py`	`plot_case_study_KPIs()`
`KPIs/`	`plot_dispatch_barchart.py`	`plot_dispatch_barchart()`
`self-sufficiency/`	`regional_self_sufficiency_level.py`	`evaluate_self_sufficiency()`
`costs/`	`analyze_total_system_cost.py`	`analyze_system_cost()`
`utilization/`	`line_usage.py`	`evaluate_line_usage()`
`Import/`	`import_analysis.py`	`analyze_imports()`
`balance_maps/`	`plot_balance_map.py`	`plot_balance_map_years()`

4.1.4 Country Scopes

Three country lists are defined in `analysis_main.py`:

Variable	Count	Countries	Used for
`countries`	33	EU + NO, GB, CH, Western Balkans	Full model scope
`eu27_countries`	25	EU27 excl. Malta, Cyprus	Network maps
`th_countries`	26	TransHyDE project countries	Import analysis

4.2 System Costs

4.2.1 Overview

Total system cost breakdown per scenario and planning year, covering capital and operational expenditure by technology group. Costs are shown both as annual values and as integrated cumulative costs from the start year to each planning horizon.

Script: `scripts_analysis/analyze_total_system_cost.py` **Function:** `analyze_system_cost()` **Output:** `costs/`

4.2.2 Output Files

File	Description
`infrastructure_capital_cost.png`	Capital costs of transmission infrastructure (electricity, H_2, gas, CO_2)
`absolute_capital_cost.png`	Absolute capital costs by technology group across all scenarios
`residual_capital_cost_{sel_scen}_{scenario}.png`	Difference in capital costs vs `sel_scen`, one file per comparison scenario
`operational_costs.png`	Operational (marginal) costs by category across all scenarios and years
`residual_operational_costs_{sel_scen}_{scenario}.png`	Difference in operational costs vs `sel_scen`, one file per comparison scenario
`total_costs.png`	Total annualised costs (capex + opex) per scenario and year
`total_overinvestment.png`	System cost difference vs `sel_scen` (bn EUR/yr)
`integrated_total_costs.png`	Cumulative total costs from start year per scenario
`integrated_difference_costs.png`	Cumulative cost difference vs `sel_scen` (bn EUR)
`total_costs.xlsx`	Excel with annualised and integrated costs by category

sel_scen

`sel_scen` is the primary scenario selected in `analysis_main.py`. It is used as the reference baseline for all residual and difference plots. Files containing `{sel_scen}` and `{scenario}` in their name are generated once per comparison scenario.

The Excel file is also automatically written into `KPIs/Summary-Results-KPIs.xlsx` if that file already exists.

4.2.3 Technology Mapping

Cost components are mapped to groups via:

```
scripts_analysis/Mapping_system_costs.csv
```

The groups used in plots are:

Group	Includes
`infrastructure`	AC/DC lines, H_2/gas/CO_2 pipelines
`power generation`	Solar, wind, nuclear, hydro, etc.
`heat generation`	Heat pumps, boilers, CHPs
`transformation`	Electrolysers, methanisation, Fischer-Tropsch, etc.
`import (capex)`	Import infrastructure capital costs
`import (opex)`	Import fuel operational costs
`operational costs`	Fuel costs, VOM
`others (capex/opex)`	Remaining components

4.2.4 Backup Capacities

`backup_capas` is hardcoded in `analysis_main.py` (currently `True`). When `True` and the backup cost file exists, gas turbine backup capacity costs are added to `total_overinvestment.png` and `integrated_difference_costs.png`. If the file is missing, the flag is automatically set to `False` by the script.

The costs are read from:

```
{analysis_results_dir}/Balances/gas_turbine_backup_costs.xlsx
```

where `{analysis_results_dir}` is the dated output folder (e.g., `analysis_results_20260311/`).

> **Note**
>
> This file is produced by `get_standard_balances()` for the `gas` carrier. The Energy Balances analysis must run before System Costs when `backup_capas=True`.

4.3 Energy Balances

4.3.1 Overview

Supply and demand balance charts are produced for 6 carriers across three regions (all countries, EU27, and one configurable country) per scenario and year. Results are saved as PNG bar charts, interactive HTML timeseries, and Excel files.

Script: `scripts_analysis/configurable_energy_balances.py` **Function:** `get_standard_balances()` **Output:** `Balances/`

4.3.2 Carriers

Carrier	Description
`electricity`	High-voltage electricity
`gas`	Natural gas / methane
`low voltage`	Residential electricity distribution
`H2`	Hydrogen
`heat`	Decentral heat
`urban central heat`	District heating

4.3.3 Output Files

For each carrier, the following are produced under `Balances/` :

File / Folder	Description
`balance_{carrier}.xlsx`	Excel with supply/demand data per country and scenario
`gas_turbine_backup_capacities.xlsx`	Gas turbine backup capacity data (electricity carrier only)
`gas_turbine_backup_costs.xlsx`	Gas turbine backup cost data (electricity carrier only)
`bar_plots/`	PNG bar charts per region and scenario mode
`timeseries_plots_line_chart/`	Interactive HTML line charts (selectable scenario/year)
`timeseries_plots_stacked_bar_chart/`	Interactive HTML stacked bar chart + PNG per scenario/year

Bar plots are generated for three regions: **All** (33 countries), **EU27**, and the configured country (default `DE`).

4.3.4 Configuration

Country

`country_to_plot` is set in `analysis_main.py` (currently `"DE"`). Any ISO2 country code in the model scope can be used.

Scenario comparison mode

`compare_scenarios` controls whether all scenarios are overlaid in one plot (`True`) or plotted individually (`False`). In `analysis_main.py` it is set to `[True, False]` for all carriers, producing both plot styles. `urban central heat` only uses `[True]` because it is not meaningful to plot it per individual scenario.

4.4 KPIs

4.4.1 Overview

Five functions produce key performance indicators covering installed capacity, infrastructure volume, dispatch, and summary system metrics.

4.4.2 Installed Capacity

Script: `installed_capacity.py` **Function:** `call_installed_capacity_plot()` **Output:** `KPIs/`

Installed capacity by technology, planning year, and scenario. Controlled by `plot_backup_capas` (boolean) in `analysis_main.py` to optionally include backup capacity.

Output files are produced for two scopes (`EU27` and `All`) and three capacity types (`generation`, `heat`, `storage`):

File	Description
`installed_{type}_capacities_{scope}.png`	All-scenario comparison chart
`installed_{type}_capacities_{scope}_{scenario}.png`	Per-scenario chart across years

4.4.3 Infrastructure Volume (TW·km)

Script: `plot_GWkm.py` **Function:** `plot_TWkm_all_carriers()` **Output:** `KPIs/`

Installed capacity × line length (TW·km) per carrier (electricity, H_2, CO_2, gas) across years and scenarios. Plots are produced for four regions: all modelled countries, EU27, Benelux, and the Baltic cluster.

File	Description
`TWkm_elec_{region}.png`	Electricity TW·km
`TWkm_H2_{region}.png`	Hydrogen TW·km
`TWkm_CO2_{region}.png`	CO_2 TW·km
`TWkm_gas_{region}.png`	Gas TW·km

Regions: `europe`, `eu27`, `benelux`, `balticum`.

4.4.4 Summary KPIs

Script: `summary_KPIs.py` **Function:** `start_KPI_analysis()` **Output:** `KPIs/`

Comprehensive Excel workbook covering total costs, emissions, and system metrics per country, EU27, and selected nodes, across all scenarios and years.

File	Description
`Summary-Results-KPIs.xlsx`	Full KPI workbook

4.4.5 Case Study KPIs

Script: `plot_cases_KPIs.py` **Function:** `plot_case_study_KPIs()` **Output:** `KPIs/`

Bar charts (with cluster map insets) comparing offshore capacity, H_2 storage, CO_2 storage, and H_2 import across scenarios and years for two predefined regional clusters: `northsea` and `balticum`.

File	Description
`offshore_capacity_{cluster}.png`	Offshore installed capacity
`H2_storage_{cluster}.png`	H_2 storage capacity
`CO2_storage_{cluster}.png`	CO_2 storage capacity
`H2_import_{cluster}.png`	H_2 import (North Sea cluster only)

4.4.6 Dispatch Barchart

Script: `plot_dispatch_barchart.py` — `plot_dispatch_barchart()`

Stacked bar chart of flexibility dispatch (storage, demand response, etc.) across scenarios and years. The `"flexibility"` argument selects the KPI type computed inside the script.

File	Description
`flexibility_dispatch.png`	Flexibility dispatch comparison

4.5 Import Analysis

4.5.1 Overview

Energy import volumes (TWh) per import technology across planning years and scenarios.

Script: `scripts_analysis/import_analysis.py` **Function:** `analyze_imports()` **Output:** `Import/`

4.5.2 Import Sets

Three sets of import carriers are evaluated independently:

Set	Carriers
`H2_import`	H_2 pipeline, H_2 shipping
`H2_derivate_import`	H_2 pipeline, H_2 shipping, syngas pipeline, syngas shipping, synfuel
`energy_import`	All of the above + coal, oil, uranium, LNG gas, pipeline gas

4.5.3 Output Files

For each import set the following files are saved to `Import/`:

File	Description
`plot_{scenario}_{set}.png`	Stacked bar chart per scenario across years
`plot_all_{set}.png`	Side-by-side comparison of all scenarios
`imports_{scenario}_TWh.xlsx`	Detailed time-series data per scenario

4.6 Balance Maps

4.6.1 Overview

Spatial maps of energy balances overlaid on the European grid, produced for each carrier, scenario, and year. A second set of difference maps compares the selected reference scenario against all other scenarios.

Script: `scripts_analysis/plot_balance_map.py` **Function:** `plot_balance_map_years()` **Output:** `balance_maps/`

4.6.2 Carriers

Carriers are defined in `config/config.plotting.yaml` under `constants.kinds`:

Carrier	Description
`carbon`	Carbon flows (CO_2 capture, stored and sequestered)
`co2`	CO_2 emissions
`electricity`	Transmission level electricity (AC and DC)
`gas`	Natural gas / methane
`hydrogen`	Hydrogen

4.6.3 Output Files

For each carrier, scenario, and year one PNG is saved to `balance_maps/`:

File	Description
`{carrier}_{region}_balance_map_{scenario}_{year}.png`	Single-scenario balance map
`{carrier}_{region}_balance_map_{scen1} vs {scen2}_{year}.png`	What is additional in `scen1` relative to `scen2`
`{carrier}_{region}_balance_map_{scen2} vs {scen1}_{year}.png`	What is additional in `scen2` relative to `scen1`

4.6.4 Configuration

Region

`country_to_plot_networks` in `analysis_main.py` controls which region is plotted. It is set to `"EU27"` by default. Three regions have dedicated plot configurations in `config.plotting.yaml`:

Value	Description
`"EU27"`	EU-27 + neighbouring countries (default)
`"PL_and_Baltics"`	Poland and the Baltic states (not plotted)
`"Benelux"`	Belgium, Netherlands, Luxembourg (not plotted)

Any other value falls back to the `"EU27"` configuration.

Reference scenario

`sel_scen` is the reference scenario used for the difference maps. Each other scenario in `scenarios` is compared against it.

> **Note**
>
> `sel_scen` is set in `analysis_main.py` and shared with other analysis functions (system costs, self-sufficiency).

5. Release Notes

- Enable download of documentation as PDF using `mkdocs-to-pdf` and enable documentation preview in PRs PR #14
- Deploy documentation and add documentation preview PR #7
- Add documentation PR #5
- Apply `snakemake` fix to prevent "another PID running" gurobi issue PR #3
- Integrate OETC PR #1